

Abstract:

Reinforcement learning (RL) lacks formal safety guarantees crucial for real-world deployment, due to the semantic gap between **formal safety models** and **stochastic RL environments**. Manually translating safety logic is laborious and error-prone. We propose a formal specification language based on differential dynamic logic, extended with annotations, to automatically synthesize safe RL environments and reward signals. This framework enables RL agents to be trained on formally proven safety specifications, with reduced reward engineering effort.

Introduction:

- **Motivation:** RL excels but lacks formal safety guarantees.
- **Gap:** Semantic disconnect between formal safety models and RL environments; manual translation is complex and error-prone.
- **Goal:** Bridge this gap with a safety model parser, enabling RL agents to be trained with formally proven safety specifications.

Proposed Framework:

Formal Specification Language: Uses differential dynamic logic to specify safety models.

Annotated Formal Models: Extends models with comments (@progress, @safety) to link formal properties to RL reward/penalty signals.

Example: Rewarding Progress (Speed)

```
// @progress: reward=isEfficient, on_success=continue
Bool isEfficient(Real v) <-> (v > 0);
```

Example: Enforcing Safety Distance

```
// @safety: penalty=-1000, shape=isSafe, on_failure=truncate
Bool isSafe(Real x, Real xo, Real v) <-> (abs(x-xo) > stopDist(v));
```

Automated Environment Generation: Automatically generates Gymnasium-compatible RL environments and reward signals from annotated specifications.

Reduced Reward Engineering: Agents learn behaviors inherently satisfying formally verified safety properties, streamlining reward function design.

Current Status & Challenges:

Progress: Successfully parsed formal models into a Gymnasium environment.

Current Issues: Challenges with initial environment variable generation and diverse sample generation.

Solution:

- **Sample Ranges:** // @var_meta: range=[0.0,50.0] annotations define variable ranges for guided parsing.
- **Z3 Soft Constraints:** Z3 with soft constraints, guided by random targets, ensures diverse, precondition-satisfying samples for training.

Future Directions:

- Develop robust solutions for initial environment variable generation to ensure valid starting states.
- Implement advanced sampling techniques to reliably generate data that satisfies model preconditions.
- Further refine the automated synthesis pipeline for broader applicability across various complex systems and RL algorithms.
- Conduct extensive evaluations to demonstrate the performance and safety benefits of RL agents trained with this framework.

References:

Stefan Mitsch. Towards Verification-Driven Control Learning *In ICAA 2024*.

Marian Qian, Stefan Mitsch. Reward Shaping from Hybrid Systems Models in Reinforcement Learning. *In NFM 2023*.

Stefan Mitsch, Andre Platzer. ModelPlex: Verified runtime validation of verified cyber-physical system models. *Formal Methods in System Design*, 49(1), pp. 33-74. 2016

Nathan Fulton, Stefan Mitsch, Jan-David Quesel, Marcus Völz, André Platzer. KeYmaera X - An Axiomatic Tactical Theorem Prover for Hybrid Systems *In CADE 2015*.

Mark Towers et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv:2407.17032

Preliminary results:

Agent State Evolution Over Time:

The accompanying plot illustrates an agent's state evolution (x, y, v, w, a) over time, generated from a formal model using our parser. This demonstrates the initial framework's capability to simulate agent behavior based on formal specifications, serving as a working prototype for further analysis.

Contacts

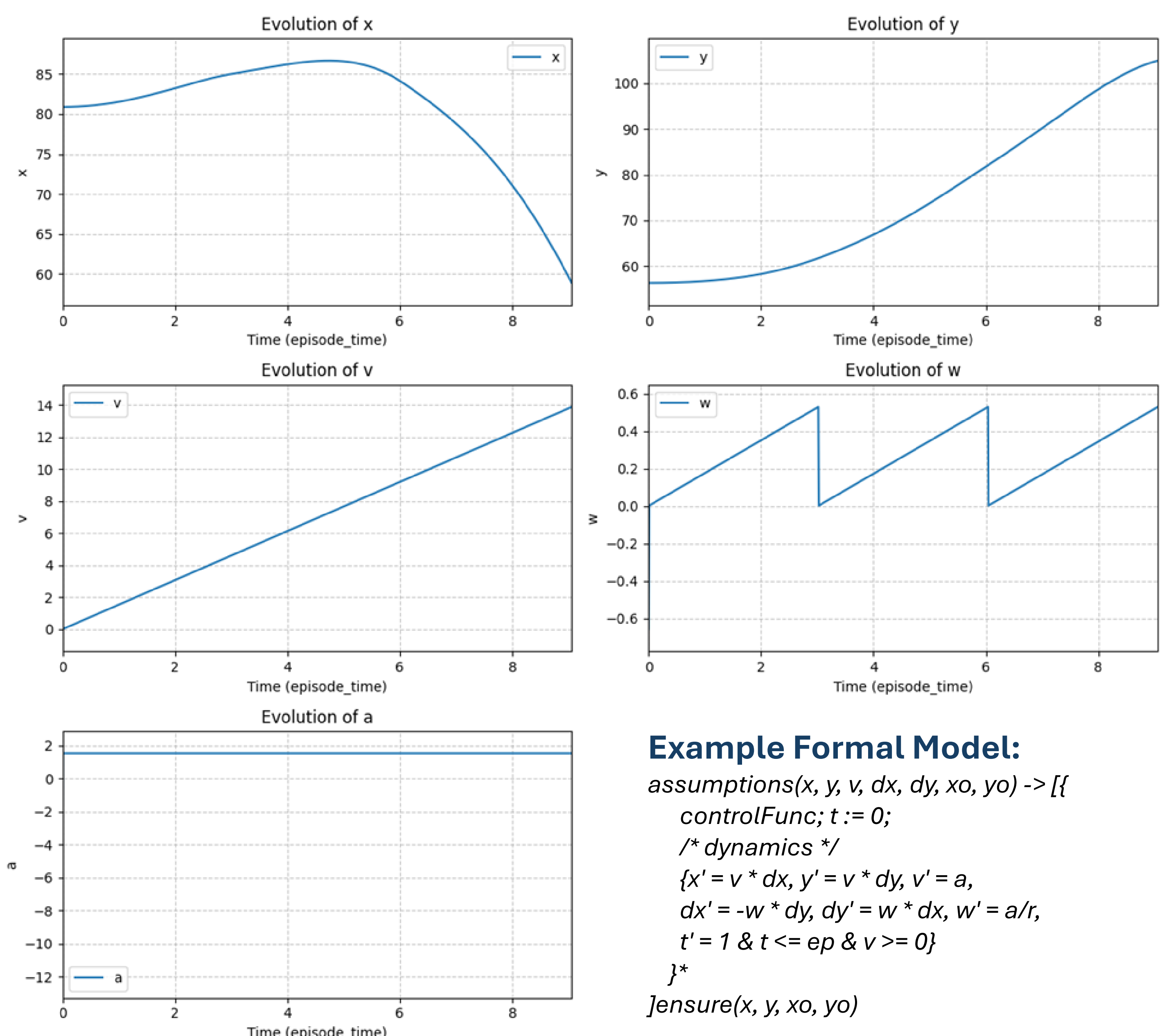
Thuan T. Cao: tcao8@depaul.edu

Stefan Mitsch: smitsch@depaul.edu

Acknowledgements:

This work was partially funded by the National Science Foundation (NSF) under Award Number 2220311.

Agent State Evolution Over Time



Example Formal Model:

```
assumptions(x, y, v, dx, dy, xo, yo) -> [{
  controlFunc; t := 0;
  /* dynamics */
  {x' = v * dx, y' = v * dy, v' = a,
   dx' = -w * dy, dy' = w * dx, w' = a/r,
   t' = 1 & t <= ep & v >= 0}
}]
jensure(x, y, xo, yo)
```